

INFORME DE LABORATORIO

Laboratorio 2024 - Script para Listar Archivos
Administración de Infraestructuras
Israel Mauricio Bellizzi Caballero



Ana Victoria Reyes Macedo
26 de Septiembre del 2024



Índice

Índice	2
Introducción	4
Gestión del Laboratorio	4
Ciclo de Vida	4
Análisis	5
Especificación de Requisitos del Software (SRS)	5
-s (Archivo Específico)	5
-g (3 Archivos Más Grandes)	5
-p (3 Archivos Más Pequeños)	5
-x (Archivos Ejecutables)	6
Requisitos de Usuario	6
Casos de Uso	6
Diseño	7
Diseño de Arquitectura del Script	7
info_archivo	7
listar_archivos	7
mostrar_uso	7
Especificación de la Interfaz	8
Salidas por pantalla	8
Manejo de Errores	10
Verificar si el archivo o directorio existe	10
Manejo de parámetros correctos	11
Controlar que los parámetros no se repitan	13
Validaciones adicionales	13
Implementación	14
Guía del Programador	14
Comandos utilizados y su justificación	14
Resumen	16
Explicación del script paso a paso	17
Plan de Pruebas	20
Testing	21
Informe de Pruebas	21
Resumen General	21
Pruebas Realizadas	21
Solución Implementada Para los Errores	22
Modificaciones	23
Manual de Usuario	23
Requisitos	23

Instalación	23
Uso	23
Errores comunes	24
Proceso de Desarrollo del Script	24
Conclusiones y Reflexiones	24
Referencias Bibliográficas	25
Anexo	25
Script completo	25

Introducción

El script **"AI_2024_list.sh"** ha sido desarrollado para simplificar la gestión de archivos y directorios en sistemas Linux, brindando una interfaz de línea de comandos intuitiva y fácil de usar. Este script permite al usuario obtener información detallada sobre archivos específicos, así como listar archivos según distintos criterios, como los más grandes, los más pequeños o los ejecutables dentro de un directorio. Con estas funciones, **"AI_2024_list.sh"** optimiza la administración de grandes volúmenes de archivos, facilitando la organización.

Gestión del Laboratorio

Ciclo de Vida

Este laboratorio sigue una metodología en cascada, adaptada a un enfoque ágil, que permite incorporar mejoras continuas en cada fase de desarrollo:

1. **Fase de Requisitos:** Identificación y recopilación de los objetivos clave del script, siguiendo las especificaciones proporcionadas por el profesor para asegurar que cumpla con las expectativas del laboratorio.
2. **Fase de Diseño:** Creación de la arquitectura del script el cual detalla la estructura y comportamiento del programa, asegurando una implementación clara y eficiente.
3. **Fase de Implementación:** Programación del script y desarrollo de la interfaz de línea de comandos, basándose en el diseño previamente definido. En esta etapa se asegura que el código cumpla con los requisitos funcionales.
4. **Fase de Pruebas:** Realización de pruebas exhaustivas para validar el funcionamiento correcto del script, identificando y corrigiendo errores potenciales, y optimizando el rendimiento.
5. **Fase de Documentación y Entrega:** Elaboración de la documentación para el usuario y el programador, incluyendo guías detalladas de uso.

Análisis

Este script en Bash, debe cumplir con una serie de funcionalidades relacionadas con la obtención de información de archivos utilizando el comando **ls** como fuente principal. Aquí una breve explicación en detalle lo que debe hacer el script, basado en los parámetros que acepta:

Fuente de Información: La fuente principal será el comando **ls**, que permite listar archivos y directorios en un sistema Linux, además de proporcionar información sobre ellos (como permisos, tamaños, dueño, etc.).

Nombre del Script: El script se llamará **AI_2024_list.sh** y se ejecutará desde la línea de comandos, aceptando distintos parámetros.

Especificación de Requisitos del Software (SRS)

-s (Archivo Específico)

Función: Cuando se pasa el parámetro **-s** junto con un archivo, se debe listar información detallada de ese archivo específico.

Datos a Mostrar:

- Nombre del archivo: Ej. **archivo.txt**
- Tamaño: En kilobytes, Ej. **148 KB**
- Dueño: Propietario del archivo, Ej. **usuario**
- Permisos: Formato **rw-rw-rw** (lectura, escritura, ejecución para usuario, grupo y otros)

Nota: El script debe manejar rutas absolutas o relativas.

-g (3 Archivos Más Grandes)

Función: Este parámetro hace que se listen los **3 archivos más grandes** dentro de un directorio específico (o el actual si no se proporciona una ruta).

La información que debe listar para cada archivo es: **Tamaño, Nombre, Dueño, Permisos**.

-p (3 Archivos Más Pequeños)

Función: Similar a -g, pero listará los **3 archivos más pequeños** en el directorio.

También mostrará la información: **Tamaño, Nombre, Dueño, Permisos.**

-x (Archivos Ejecutables)

Función: Este parámetro filtra los archivos para mostrar solo aquellos que son **ejecutables** (es decir, aquellos que tienen permiso de ejecución, x en la columna de permisos).

Combinación: Puede combinarse con los parámetros **-g** o **-p** para mostrar los archivos más grandes o pequeños que sean ejecutables.

Requisitos de Usuario

El script debe ejecutarse en sistemas basados en Linux: El script está diseñado para sistemas operativos basados en Linux, que utilizan comandos comunes en entornos Unix. Aunque podría ejecutarse en Windows mediante WSL (Windows Subsystem for Linux), se recomienda su uso en distribuciones Linux como Ubuntu, CentOS o Fedora.

Permisos de Ejecución: El archivo del script debe tener permisos de ejecución para poder ser ejecutado desde la línea de comandos.

Casos de Uso

Caso 1: El usuario proporciona un archivo específico para obtener información detallada utilizando la opción **-s**.

Caso 2: El usuario lista los tres archivos más grandes de un directorio con la opción **-g**.

Caso 3: El usuario lista los tres archivos más pequeños de un directorio utilizando la opción **-p**.

Caso 4: El usuario filtra y muestra solo los archivos ejecutables con la opción **-x**.

Caso 5: El usuario lista los tres archivos más grandes que son ejecutables con la opción combinada **-x -g**.

Caso 6: El usuario lista los tres archivos más pequeños que son ejecutables con la opción combinada **-x -p**.

Diseño

Diseño de Arquitectura del Script

El script sigue una arquitectura modular, con funciones dedicadas a la obtención de información, listado de archivos y validación de parámetros, lo que facilita su mantenimiento y extensión. A continuación, se describen las funciones principales:

info_archivo

Descripción: Esta función recibe un archivo como argumento y muestra su nombre, tamaño, propietario y permisos. Utiliza comandos como `ls`, `awk`, y `cut` para extraer y formatear la información. La salida se presenta en el siguiente formato:

Nombre: archivo.txt

Tamaño: 148 kb

Dueño: usuario

Permisos: `rwxr-xr-x`

listar_archivos

Descripción: Función que acepta tres parámetros: el directorio a analizar, el número de archivos a listar y el tipo de filtro (archivos ejecutables o no). Dependiendo de si se buscan archivos más grandes o más pequeños, utiliza `ls` para ordenar los archivos según su tamaño. La salida tiene el siguiente formato:

Tamaño	Nombre	Dueño	Permisos
148 kb	archivo.txt	usuario	<code>rwxrw-rw</code>
1092 kb	arch_la	usuario	<code>rwxr--r-x</code>

mostrar_uso

Descripción: Esta función no recibe parámetros y se utiliza para mostrar al usuario cómo debe ejecutar correctamente el script. Se llama en situaciones donde el usuario no proporciona los argumentos necesarios o utiliza opciones no válidas. La función imprime un mensaje de ayuda en la terminal que describe los parámetros permitidos y su sintaxis. Su formato de salida deberá ser parecido al siguiente:

Uso: `./AI_2024_list.sh [-s archivo] [-g] [-p] [-x] [ruta]`

- s archivo Especificar archivo para mostrar detalles.
- g Listar los 3 archivos más grandes.
- p Listar los 3 archivos más pequeños.
- x Mostrar solo archivos ejecutables.

Especificación de la Interfaz

Salidas por pantalla

Las salidas del script están diseñadas para ser claras y concisas. Dependiendo de los parámetros, se puede esperar un determinado formato de salida por pantalla, a continuación se van a mostrar. Como aclaración, temporalmente se borró el comando "clear" para una mejor visualización de cómo se utiliza el script y el formato que sale.

Opción -s (información de un archivo): Muestra los detalles del archivo especificado.

Figura 1 — Salida por pantalla del parámetro -s

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -s AI_2024_list.sh
Nombre: AI_2024_list.sh
Tamaño: 8,0K
Dueño: anyes
Permisos: -rwxrwxr-x
anyes@ana-reyes:~$ ./AI_2024_list.sh -s /home/anyes/Descargas/c.pdf
Nombre: c.pdf
Tamaño: 1,7M
Dueño: anyes
Permisos: -rw-rw-r--
```

Opciones -p (archivos pequeños): Lista los archivos más pequeños en el directorio especificado.

Figura 2 — Salida por pantalla del parámetro -p

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -p
```

Tamaño	Nombre	Dueño	Permisos
0	.sudo_as_admin_successful	anyes	-rw-r--r--
143	lala.cpp	anyes	-rw-rw-r--
220	.bash_logout	anyes	-rw-r--r--

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -p /home/anyes/Descargas/
```

Tamaño	Nombre	Dueño	Permisos
1,7M	c.pdf	anyes	-rw-rw-r--
1,9M	Concepto y clasificación.pdf	anyes	-rw-rw-r--
1,9M	Proyectos fracasados y exitosos.pdf	anyes	-rw-rw-r--

Opciones -g (archivos grandes): Lista los archivos más grandes en el directorio especificado.

Figura 3 — Salida por pantalla del parámetro -g

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -g
```

Tamaño	Nombre	Dueño	Permisos
13K	PracticoListas.cpp	anyes	-rw-rw-r--
13K	PracticoArboles.cpp	anyes	-rw-rw-r--
6,5K	AI_2024_list (Copiar).sh	anyes	-rwxrwxr-x

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -g /home/anyes/Descargas/
```

Tamaño	Nombre	Dueño	Permisos
1,9M	Proyectos fracasados y exitosos.pdf	anyes	-rw-rw-r--
1,9M	Concepto y clasificación.pdf	anyes	-rw-rw-r--
1,7M	c.pdf	anyes	-rw-rw-r--

Opción -x: Muestra únicamente los archivos que tienen permisos de ejecución.

Figura 4 — Salida por pantalla del parámetro -x

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -x
```

Tamaño	Nombre	Dueño	Permisos
6,2K	ojala (Copiar).sh	anyes	-rwxrwxr-x
7,2K	ojala.sh	anyes	-rwxrwxr-x
5,9K	AI_2024_list (Copiar 3).sh	anyes	-rwxrwxr-x
7,2K	AI_2024_list.sh	anyes	-rwxrwxr-x
6,5K	AI_2024_list (Copiar).sh	anyes	-rwxrwxr-x
6,5K	AI_2024_list (Copiar 2).sh	anyes	-rwxrwxr-x
6,6K	ojala (Copiar 2).sh	anyes	-rwxrwxr-x

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -x /home/anyes/Descargas/
```

Tamaño	Nombre	Dueño	Permisos
6,0K	ojala.sh	anyes	-rwxrwxr-x
5,9K	AI_2024_list (Copiar 3).sh	anyes	-rwxrwxr-x
6,5K	AI_2024_list (Copiar 2).sh	anyes	-rwxrwxr-x

Combinación de -x con otras opciones: Filtra los archivos ejecutables que son los más grandes o pequeños, según el parámetro adicional (-g o -p).

Figura 5 — Salida por pantalla del parámetro -x -p

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -x -p
```

Tamaño	Nombre	Dueño	Permisos
5,9K	AI_2024_list (Copiar 3).sh	anyes	-rwxrwxr-x
6,2K	ojala (Copiar).sh	anyes	-rwxrwxr-x
6,5K	AI_2024_list (Copiar 2).sh	anyes	-rwxrwxr-x

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -x -p /home/anyes/Descargas/
```

Tamaño	Nombre	Dueño	Permisos
5,9K	AI_2024_list (Copiar 3).sh	anyes	-rwxrwxr-x
6,0K	ojala.sh	anyes	-rwxrwxr-x
6,5K	AI_2024_list (Copiar 2).sh	anyes	-rwxrwxr-x

Figura 6 — Salida por pantalla del parámetro -x -g

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -x -g
```

Tamaño	Nombre	Dueño	Permisos
7,2K	ojala.sh	anyes	-rwxrwxr-x
7,2K	AI_2024_list.sh	anyes	-rwxrwxr-x
6,6K	ojala (Copiar 2).sh	anyes	-rwxrwxr-x

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -x -g /home/anyes/Descargas/
```

Tamaño	Nombre	Dueño	Permisos
6,5K	AI_2024_list (Copiar 2).sh	anyes	-rwxrwxr-x
6,0K	ojala.sh	anyes	-rwxrwxr-x
5,9K	AI_2024_list (Copiar 3).sh	anyes	-rwxrwxr-x

Manejo de Errores

Este script hace varias validaciones para asegurarse de que el comportamiento sea correcto y el flujo del script sea manejado adecuadamente. A continuación, se detallan las validaciones clave:

Verificar si el archivo o directorio existe

En la función **info_archivo()**, el script verifica si el archivo especificado existe con **-f "\$archivo"**. Si no existe, muestra un mensaje de error y sale.

Figura 7 — Salida por pantalla de archivo inexistente

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -s pepe.txt
ERROR: El archivo pepe.txt no existe.
Verifique la ruta o el nombre del archivo.
Uso: ./AI_2024_list.sh [-s archivo] [-g] [-p] [-x] [ruta]
-s archivo   Especificar archivo para mostrar detalles.
-g           Listar los 3 archivos más grandes.
-p           Listar los 3 archivos más pequeños.
-x           Mostrar solo archivos ejecutables.
```

En la función **listar_archivos()**, verifica si el directorio existe con **-d "\$directorio"**. Si no, también muestra un mensaje de error y sale.

Figura 8 — Salida por pantalla de directorio inexistente

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -p /pepe/lala
ERROR: La ruta: /pepe/lala no existe.
Verifique la ruta.
Uso: ./AI_2024_list.sh [-s archivo] [-g] [-p] [-x] [ruta]
-s archivo   Especificar archivo para mostrar detalles.
-g           Listar los 3 archivos más grandes.
-p           Listar los 3 archivos más pequeños.
-x           Mostrar solo archivos ejecutables.
```

Manejo de parámetros correctos

Uso incorrecto de parámetros: El script verifica si no se ha pasado ningún parámetro (con **if [\$# -eq 0]**) y, si es así, muestra un mensaje de ayuda usando la función **mostrar_uso()** y termina la ejecución.

Figura 9 — Salida por pantalla sin parámetros

```
anyes@ana-reyes:~$ ./AI_2024_list.sh
ERROR: No te olvides de pasar un parametro! :)
Uso: ./AI_2024_list.sh [-s archivo] [-g] [-p] [-x] [ruta]
-s archivo   Especificar archivo para mostrar detalles.
-g           Listar los 3 archivos más grandes.
-p           Listar los 3 archivos más pequeños.
-x           Mostrar solo archivos ejecutables.
```

Controlar parámetros válidos: El script se asegura de que no se usen los parámetros fuera de los establecidos. Si se pasan ambos, muestra un error y termina.

Figura 10 — Salida por pantalla parámetro no válido

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -n
ERROR: El parámetro '-n' no es válido.
Uso: ./AI_2024_list.sh [-s archivo] [-g] [-p] [-x] [ruta]
-s archivo   Especificar archivo para mostrar detalles.
-g           Listar los 3 archivos más grandes.
-p           Listar los 3 archivos más pequeños.
-x           Mostrar solo archivos ejecutables.
```

Controlar parámetros contradictorios: El script se asegura de que no se usen los parámetros **-g** (para listar archivos grandes) y **-p** (para listar archivos pequeños) al mismo tiempo. Si se pasan ambos, muestra un error y termina.

Figura 11 — Salida por pantalla parámetros contradictorios

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -p -g /home/anyes/Descargas/
ERROR: No se pueden usar las opciones -g y -p al mismo tiempo.
Uso: ./AI_2024_list.sh [-s archivo] [-g] [-p] [-x] [ruta]
-s archivo   Especificar archivo para mostrar detalles.
-g           Listar los 3 archivos más grandes.
-p           Listar los 3 archivos más pequeños.
-x           Mostrar solo archivos ejecutables.
```

Controlar cantidad de parámetro: El script se asegura de que no se usen más de dos parámetros al mismo tiempo. Si se pasan ambos, muestra un error y termina.

Figura 12 — Salida por pantalla parámetros de más

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -x -g -p /home/anyes/Descargas/
ERROR: Solo se permiten hasta 2 parámetros de opción al mismo tiempo.
Uso: ./AI_2024_list.sh [-s archivo] [-g] [-p] [-x] [ruta]
-s archivo   Especificar archivo para mostrar detalles.
-g           Listar los 3 archivos más grandes.
-p           Listar los 3 archivos más pequeños.
-x           Mostrar solo archivos ejecutables.
```

Falta de argumentos obligatorios: Si se usa la opción **-s** para especificar un archivo pero no se pasa un archivo después de **-s**, el script muestra un error indicando que es necesario proporcionar un archivo.

Figura 13 — Salida por pantalla falta archivo

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -s
ERROR: Debe proporcionar un archivo después de -s.
Uso: ./AI_2024_list.sh [-s archivo] [-g] [-p] [-x] [ruta]
-s archivo    Especificar archivo para mostrar detalles.
-g            Listar los 3 archivos más grandes.
-p            Listar los 3 archivos más pequeños.
-x            Mostrar solo archivos ejecutables.
```

Controlar que los parámetros no se repitan

Permite combinar **-x** (mostrar sólo archivos ejecutables) con las opciones **-g** o **-p** sin generar conflictos.

Figura 14 — Salida por pantalla x no combinable

```
anyes@ana-reyes:~$ ./AI_2024_list.sh -x -s /home/anyes/Descargas/
ERROR: La opción -s no puede combinarse con otras opciones.
Uso: ./AI_2024_list.sh [-s archivo] [-g] [-p] [-x] [ruta]
-s archivo    Especificar archivo para mostrar detalles.
-g            Listar los 3 archivos más grandes.
-p            Listar los 3 archivos más pequeños.
-x            Mostrar solo archivos ejecutables.
anyes@ana-reyes:~$ ./AI_2024_list.sh -x -y /home/anyes/Descargas/
ERROR: El parámetro '-y' no es válido.
Uso: ./AI_2024_list.sh [-s archivo] [-g] [-p] [-x] [ruta]
-s archivo    Especificar archivo para mostrar detalles.
-g            Listar los 3 archivos más grandes.
-p            Listar los 3 archivos más pequeños.
-x            Mostrar solo archivos ejecutables.
```

Validaciones adicionales

Formateo de la salida: Utiliza colores y mensajes claros para errores y resultados, mejorando la legibilidad y ayudando al usuario a identificar problemas rápidamente.

Figura 15 — Formato de salida por colores

```
# Colores para el texto usando secuencias de escape
nada='\033[00m'    # Sin color (por defecto)
rojo='\033[01;31m'  # Rojo para mensajes de error
negrita='\033[1m'   # Negrita para destacar textos
verde='\033[1;32m'  # Verde para mostrar el uso correcto
```

Estas son las validaciones y controles adicionales que el script implementa para asegurar una ejecución robusta y manejable por el usuario.

Implementación

Guia del Programador

Comandos utilizados y su justificación

ls

Uso en el script: El comando **ls** se utiliza para listar archivos y directorios, y también para obtener detalles de archivos específicos (como permisos, dueño y tamaño).

Justificación: Es una herramienta fundamental en Linux para explorar el sistema de archivos. Ofrece opciones que permiten mostrar detalles como permisos, propietario, y tamaño de los archivos, lo que es esencial para las funcionalidades del script.

La opción **ls -l** proporciona una vista detallada de los archivos, incluyendo permisos y el propietario, que es necesaria para las funciones que muestran información sobre archivos.

du

Uso en el script: El comando **du** (disk usage) se utiliza para medir el tamaño de los archivos. En el script, se usa para obtener el tamaño de un archivo específico.

Justificación: Es ideal para obtener el tamaño de archivos y directorios en un formato legible. Es más adecuado que **ls** para mostrar el tamaño real del archivo porque **ls -l** muestra el tamaño en bloques que no siempre es intuitivo para el usuario.

La opción **-h** (human-readable) convierte el tamaño en un formato más comprensible (KB, MB, GB), lo cual mejora la legibilidad de la salida del script.

find

Uso en el script: Se utiliza para buscar archivos en un directorio y sus subdirectorios. Se emplea para localizar archivos que cumplen con ciertos criterios, como ser ejecutables o para listar todos los archivos en un directorio específico.

Justificación: Es extremadamente versátil para buscar archivos basados en diversos criterios. Permite buscar archivos por tipo, permisos, tamaño y otros atributos.

En el script, **find** con **-type f** se asegura de que sólo se seleccionen archivos y no directorios. La opción **-executable** filtra archivos que tienen permisos de ejecución, y **-printf "%s %p\n"** permite personalizar el formato de salida, lo cual es útil para la posterior clasificación y filtrado de archivos.

sort

Uso en el script: Se usa para ordenar las líneas de texto. En el script, se utiliza para ordenar los archivos por tamaño, ya sea de manera ascendente (para los archivos más pequeños) o descendente (para los archivos más grandes).

Justificación: Es una herramienta eficiente para ordenar datos basados en texto, y es fundamental cuando se necesita procesar listas de archivos ordenados por tamaño.

La opción **-n** permite ordenar numéricamente, lo cual es esencial para ordenar archivos según su tamaño de manera precisa.

numfmt

Uso en el script: Se utiliza para formatear números en un formato legible. En el script, convierte tamaños de archivos en bytes a formatos más legibles como KB, MB, etc.

Justificación: Es ideal para convertir números a formatos legibles, especialmente cuando se trabaja con tamaños de archivos. Esto facilita la lectura de datos al usuario final, ya que los tamaños de archivos en formato binario o decimal son más comprensibles cuando están en KB, MB o GB.

El uso de **numfmt --to-iec** convierte los tamaños en unidades más amigables y entendibles, mejorando la presentación de la salida.

basename

Uso en el script: Se utiliza para extraer el nombre de archivo a partir de una ruta completa. En el script, se usa para obtener el nombre del archivo sin la ruta del directorio.

Justificación: Simplifica la tarea de obtener solo el nombre del archivo de una ruta completa, lo que es útil para mostrar información de archivos sin la ruta completa.

Esto facilita la presentación de datos al usuario, mostrando nombres de archivos de manera más concisa y clara.

awk

Uso en el script: Se usa para procesar y analizar texto en columnas. En el script, se emplea para extraer campos específicos de la salida de otros comandos, como permisos, dueño y tamaño de archivos.

Justificación: Es muy potente para el procesamiento de texto y es ideal para extraer y formatear columnas de datos. Permite seleccionar columnas específicas y realizar operaciones en esos datos.

En el script, **awk** se usa para obtener el dueño (**awk '{print \$3}'**) y los permisos (**awk '{print \$1}'**) de un archivo, lo que facilita la recolección de información detallada sobre archivos.

cut

Uso en el script: Se utiliza para extraer secciones de líneas de texto. En el script, se usa para extraer el tamaño del archivo de la salida del comando **du**.

Justificación: Es útil para extraer partes específicas de la salida de otros comandos. Permite seleccionar y mostrar solo la información relevante, simplificando el procesamiento de datos.

En el script, **cut** se usa para obtener solo la primera columna del resultado del comando **du**, que representa el tamaño del archivo, eliminando cualquier información adicional.

Resumen

Cada uno de estos comandos se utiliza en el script porque ofrece una funcionalidad específica y útil para las tareas de administración de archivos:

ls: proporciona información detallada de archivos.

du: mide el tamaño de los archivos en un formato legible.

find: permite buscar y filtrar archivos basados en criterios específicos.

sort: ordena archivos por tamaño de manera efectiva.

numfmt: convierte el tamaño de archivos en un formato legible y amigable para el usuario.

basename: es utilizado para extraer y mostrar el nombre de un archivo sin la ruta completa, simplificando la visualización de nombres de archivos en la salida del script.

awk: permite procesar y extraer columnas específicas de texto, facilitando la obtención de detalles como permisos y dueño de archivos a partir de la salida de otros comandos.

cut: se utiliza para extraer partes específicas de la salida de texto, como el tamaño del archivo, para que sea más fácil de manejar y presentar.

Estos comandos proporcionan una combinación efectiva para administrar y mostrar información de archivos en el script **AI_2024_list.sh**. Cada uno aporta funcionalidades específicas que ayudan a lograr el objetivo de listar y procesar archivos de manera eficiente.

Explicación del script paso a paso

1. **Encabezado del Script:** Este encabezado proporciona contexto sobre el propósito del script y quién lo escribió. Se usa `#!/bin/bash` para especificar el intérprete que se utilizará, lo cual es estándar en la mayoría de los scripts Bash.

```
#!/bin/bash
# Autora: Ana Reyes
# Fecha: 13 - 09 - 2024
# AI_2024_list.sh - Script para listar información de archivos basado en ciertos parámetros.
```

2. **Limpieza de pantalla:** Comienza limpiando la pantalla, lo que garantiza que la salida del script se presente de manera limpia sin interferencia de comandos previos.
3. **Definición de colores y formato:** Aquí se definen secuencias de escape ANSI para agregar color y estilo a la salida del script. El uso de colores mejora la experiencia del

usuario, destacando los mensajes importantes como errores en rojo, instrucciones en verde y énfasis con negrita. Esto lo aprendí en UTU, siendo una gran recomendación por parte de mis profesores para ayuda tanto para desarrolladores como usuarios finales.

```
# Colores para el texto usando secuencias de escape
nada='\033[00m' # Sin color (por defecto)
rojo='\033[01;31m' # Rojo para mensajes de error
negrita='\033[1m' # Negrita para destacar textos
verde='\033[1;32m' # Verde para mostrar el uso correcto
```

4. **Función para mostrar el uso correcto:** La función `mostrar_uso` sirve para mostrar al usuario cómo usar el script correctamente. Se utiliza si el usuario no proporciona argumentos o los proporciona de manera incorrecta. Incluir instrucciones claras en el script facilita que los usuarios comprendan cómo interactuar con él y corrige posibles malentendidos antes de que ocurran errores más graves.

```
# Función para mostrar el uso del script en caso de uso incorrecto
mostrar_uso() {
    echo -e "${verde}Uso: $o [-s archivo] [-g] [-p] [-x] [ruta]${nada}"
    echo " -s archivo Especificar archivo para mostrar detalles."
    echo " -g      Listar los 3 archivos más grandes."
    echo " -p      Listar los 3 archivos más pequeños."
    echo " -x      Mostrar solo archivos ejecutables."
    exit 1
}
```

5. **Verificación de la cantidad de parámetros:** Si no se proporcionan argumentos, el script muestra un error y la función `mostrar_uso`. Esta verificación inicial evita que el script continúe ejecutándose sin los parámetros necesarios. Prevenir errores tempranos con una verificación de argumentos es fundamental para evitar errores no deseados más adelante.

```
# Verificar si no se han pasado parámetros al script
if [ $# -eq 0 ]; then
    echo -e "${rojo}ERROR:${nada} No te olvides de pasar un parametro! :)"
    mostrar_uso
fi
```

- 6. Función para obtener detalles de un archivo:** Esta función toma un archivo como argumento y muestra detalles como su tamaño, propietario y permisos. Antes de procesar, verifica si el argumento es un archivo válido y no un directorio. Esta verificación es necesaria para evitar errores comunes de entrada y garantiza que se procese solo lo que se espera. Los comandos `du` y `ls` son eficientes para obtener información de archivos.

```
info_archivo() {  
    archivo=$1  
    if [ -d "$archivo" ]; then  
        echo -e "${rojo}ERROR:${nada} El parámetro -s solo acepta archivos, no directorios."  
        mostrar_uso  
    fi  
    if [ ! -f "$archivo" ]; then  
        echo -e "${rojo}ERROR:${nada} El archivo ${negrita}$archivo${nada} no existe."  
        echo -e Verifique la ${negrita}ruta${nada} o el ${negrita}nombre${nada} del archivo.  
        mostrar_uso  
    fi  
    nombre=$(basename "$archivo")  
    tamaño=$(du -h "$archivo" | cut -f1)  
    dueno=$(ls -l "$archivo" | awk '{print $3}')  
    permisos=$(ls -l "$archivo" | awk '{print $1}')  
    echo -e "${negrita}Nombre:${nada} $nombre"  
    echo -e "${negrita}Tamaño:${nada} $tamaño"  
    echo -e "${negrita}Dueño:${nada} $dueno"  
    echo -e "${negrita}Permisos:${nada} $permisos"  
}
```

- 7. Función para listar archivos:** Esta función lista los archivos más grandes o pequeños dentro de un directorio, también, los que son ejecutables si se combina con el parámetro `-x`. Se utilizan los comandos `find` para encontrar archivos y `sort` para ordenarlos. `find` es una herramienta flexible para buscar archivos basados en distintos criterios (tamaño,

permisos, etc.). En este caso, permite filtrar archivos ejecutables o simplemente listar archivos de acuerdo a su tamaño, lo cual hace que el script sea muy flexible.

```
listar_archivos() {  
    directorio=$1  
    cantidad=$2  
    filtro=$3  
    # Verificaciones de errores y listado de archivos  
}
```

8. **Manejo de parámetros y bucle principal:** Este bucle itera sobre los argumentos proporcionados al script, identificando cada opción (-s, -g, -p, -x) y actuando en consecuencia. Usar un bucle permite manejar múltiples parámetros de manera eficiente y flexible. Además, se asegura de que las combinaciones prohibidas (como -g y -p juntas) sean detectadas y reportadas correctamente.

```
while [ "$i" -le "$#" ]; do  
    arg=${!i}  
    # Evaluación de cada parámetro  
done
```

Plan de Pruebas

Se realizaron pruebas en los siguientes escenarios:

1. **Sin parámetros:** Verificar el manejo de errores y la visualización de la ayuda.
2. **Opción -s:** Probar con archivos existentes y no existentes para validar la obtención de detalles y gestión de errores.
3. **Opciones -g y -p:** Comprobar en directorios con distintos tamaños de archivos para verificar el orden correcto (ascendente/descendente).
4. **Opción -x:** Asegurar que solo se filtren archivos ejecutables.
5. **Combinaciones de -x con -g o -p:** Verificar que los archivos listados sean ejecutables y cumplan con los criterios de tamaño.

Testing

Informe de Pruebas

Resumen General

Las pruebas realizadas al script "**AI_2024_list.sh**" han confirmado que, en general, el script maneja adecuadamente los casos de uso previstos y responde correctamente a los errores. Sin embargo, se identificaron algunas limitaciones en el funcionamiento de ciertos parámetros, específicamente con la opción **-x**.

Pruebas Realizadas

Ejecución sin Parámetros:

- **Descripción:** Se ejecutó el script sin proporcionar ningún parámetro.
- **Resultado:** El script mostró correctamente un mensaje de ayuda indicando el uso adecuado del mismo.

Uso de la Opción -s:

- **Descripción:** Se probó la opción -s con un archivo existente y uno que no existía.
- **Resultado:** El script devolvió la información del archivo especificado correctamente en el caso de un archivo existente. En el caso del archivo no existente, se mostró un mensaje de error claro, indicando que el archivo no fue encontrado.

Uso de la Opción -g:

- **Descripción:** Se probó la opción -g para listar los tres archivos más grandes en un directorio con varios archivos.
- **Resultado:** El script listó correctamente los archivos más grandes, mostrando tamaño, nombre, dueño y permisos.

Uso de la Opción -p:

Descripción: Se utilizó la opción -p para listar los tres archivos más pequeños en un directorio.

Resultado: El script funcionó como se esperaba, mostrando correctamente los archivos más pequeños en el directorio especificado.

Uso de la Opción -x (solo):

- **Descripción:** Se ejecutó el script con la opción -x por sí sola para intentar listar solo archivos ejecutables.
- **Resultado:** No se mostraron resultados. El script no devolvió información alguna, lo que generó confusión sobre su funcionamiento.

Uso de la Opción -x combinada con -g:

- **Descripción:** Se probó la opción -x en combinación con -g para listar los archivos más grandes que sean ejecutables.
- **Resultado:** El script funciona correctamente, mostrando solo los archivos ejecutables más grandes, confirmando que la combinación de parámetros permite al script operar como se espera.

Uso de la Opción -x combinada con -p:

- **Descripción:** Se utilizó la opción -x junto con -p para listar los archivos más pequeños que sean ejecutables.
- **Resultado:** Nuevamente, el script funciona correctamente, listando solo los archivos ejecutables más pequeños, lo que sugiere que la funcionalidad de filtrado está activa solo en combinación con otras opciones.

Solución Implementada Para los Errores

El problema estaba en cómo se gestiona la opción -x sin que esté combinada con -g o -p. Cuando se usaba solo -x, el script no estaba manejando adecuadamente ese caso porque seguía esperando que también se seleccionara uno de los modos de tamaño.

Lo que se necesita es que, si solo se pasa -x el script lista todos los archivos ejecutables en el directorio, sin importar su tamaño. Se realizaron algunos ajustes al script para que también contemple ese caso.

Modificaciones

1. Agregar una verificación para la opción -x sin -g ni -p.
2. Si solo se pasa -x, se debe listar todos los ejecutables sin limitar por tamaño.

Segunda prueba -x (solo):

- **Descripción:** Se ejecutó el script con la opción -x por sí sola para intentar listar sólo archivos ejecutables.
- **Resultado:** El script ahora funciona correctamente, listando solo los archivos ejecutables sin importar el tamaño.

Manual de Usuario

Requisitos

Tener un sistema operativo basado en Linux.

Permisos de ejecución sobre el archivo del script.

Instalación

Descargar el archivo **AI_2024_list.sh** y dar permisos de ejecución con el siguiente comando:

```
chmod +x AI_2024_list.sh.
```

Uso

Para obtener detalles de un archivo específico:

```
./AI_2024_list.sh -s archivo.txt o ./AI_2024_list.sh -s /ruta/archivo.txt
```

Para listar los tres archivos más grandes en un directorio:

```
./AI_2024_list.sh -g o ./AI_2024_list.sh -g /ruta/directorio
```

Para listar los tres archivos más pequeños:

```
./AI_2024_list.sh -p o ./AI_2024_list.sh -p /ruta/directorio
```

Para mostrar sólo archivos ejecutables:

```
./AI_2024_list.sh -x o en combinación con -g o -p como ./AI_2024_list.sh -x -g
```

Errores comunes

Archivo o directorio no existente: Asegúrese de proporcionar rutas válidas.

Combinación de parámetros inválida: No se puede usar -g y -p al mismo tiempo.

Proceso de Desarrollo del Script

El laboratorio del script comenzó el 6 de septiembre de 2024, cuando se proporcionó la consigna. El análisis y diseño inicial del script comenzó el 11 de septiembre de 2024, con el objetivo de encontrar una solución sencilla. Durante esta etapa, se llevó a cabo una pequeña investigación para seleccionar los comandos más adecuados que se ajustaran a los requisitos específicos del script.

El 13 de septiembre de 2024, se inició la primera fase de implementación, siguiendo las pautas definidas en el análisis y diseño. La implementación total del script requirió aproximadamente 12 horas. Al revisar una clase en línea, noté la ausencia de algunas validaciones y la necesidad de un formato de salida específico en forma de tabla, lo que llevó a ajustes adicionales.

El 15 de septiembre de 2024, se realizaron las primeras secciones de la documentación y se llevaron a cabo pruebas exhaustivas del script, con el objetivo de identificar fallas y corregirlas de manera rápida. Finalmente, el 21 de septiembre de 2024, se retomó la documentación y se realizaron nuevas pruebas para asegurar el correcto funcionamiento del script, completando así el laboratorio en su totalidad.

Conclusiones y Reflexiones

Este script es una herramienta sencilla pero potente para la administración de archivos en sistemas Linux. Permite a los usuarios listar rápidamente archivos por tamaño y filtrar aquellos que son ejecutables, lo cual es particularmente útil en situaciones donde se necesita revisar archivos de gran volumen o encontrar archivos ejecutables en directorios complejos.

El código está modularizado, con funciones que separan claramente las responsabilidades del script, lo que facilita la expansión y el mantenimiento. Además, se han implementado

controles robustos para garantizar una buena experiencia de usuario, incluso ante errores comunes como archivos inexistentes o parámetros inválidos.

En conclusión, este laboratorio no sólo permitió profundizar en el uso de comandos avanzados de Linux, sino también en la importancia de una buena gestión. Las lecciones aprendidas de este proyecto, especialmente sobre la importancia de las pruebas exhaustivas y la documentación clara, son aplicables a futuros laboratorios o proyectos, tanto en términos de desarrollo técnico como de manejo de la planificación y ejecución.

Referencias Bibliográficas

awk: <https://geekland.eu/uso-del-comando-awk-en-linux-y-unix-con-ejemplos/>

find: <https://www.ionos.com/digitalguide/server/configuration/linux-find-command/>

Comandos básicos de Linux: <https://www.gnu.org/software/bash/manual/bash.html>

numfmt: <https://man7.org/linux/man-pages/man1/numfmt.1.html>

basename: <https://blog.carrerainlinux.com.ar/2018/04/basename-y-dirname/>

Anexo

Script completo

```
#!/bin/bash
# Autora: Ana Reyes
# Fecha: 13 - 09 - 2024
# AI_2024_list.sh - Script para listar información de archivos basado en
ciertos parámetros.
# El script acepta los siguientes parámetros:
# -s: Especificar un archivo para obtener información detallada.
# -g: Listar los 3 archivos más grandes en un directorio.
# -p: Listar los 3 archivos más pequeños en un directorio.
# -x: Mostrar solo archivos ejecutables (se puede combinar con -g o -p).

# Limpiar la pantalla
clear
```

```
# Colores para el texto usando secuencias de escape
nada='\033[00m'      # Sin color (por defecto)
rojo='\033[01;31m'   # Rojo para mensajes de error
negrita='\033[1m'     # Negrita para destacar textos
verde='\033[1;32m'   # Verde para mostrar el uso correcto

# Función para mostrar el uso del script en caso de uso incorrecto
mostrar_uso() {
    echo -e "${verde}Uso: $0 [-s archivo] [-g] [-p] [-x] [ruta]${nada}"
    echo "  -s archivo    Especificar archivo para mostrar detalles."
    echo "  -g            Listar los 3 archivos más grandes."
    echo "  -p            Listar los 3 archivos más pequeños."
    echo "  -x            Mostrar solo archivos ejecutables."
    exit 1
}

# Verificar si no se han pasado parámetros al script
if [ $# -eq 0 ]; then
    # Si no se han proporcionado parámetros, muestra un mensaje de error y la
    # ayuda del uso del script
    echo -e "${rojo}ERROR:${nada} No te olvides de pasar un parametro! :)"
    mostrar_uso
fi

# Función para obtener información detallada de un archivo específico
info_archivo() {
    archivo=$1
    # Verifica si el archivo es un directorio, lo cual es inválido para la
    # opción -s
    if [ -d "$archivo" ]; then
        echo -e "${rojo}ERROR:${nada} El parámetro -s solo acepta archivos, no
        directorios."
        mostrar_uso
    fi
}
```

```
# Verifica si el archivo no existe
if [ ! -f "$archivo" ]; then
    echo -e "${rojo}ERROR:${nada} El archivo ${negrita}$archivo${nada} no
existe."
    echo -e Verifique la ${negrita}ruta${nada} o el ${negrita}nombre${nada}
del archivo.
    mostrar_uso
fi

# Si el archivo existe, obtiene y muestra su información: nombre, tamaño,
dueño y permisos
nombre=$(basename "$archivo")
tamano=$(du -h "$archivo" | cut -f1)
dueno=$(ls -l "$archivo" | awk '{print $3}')
permisos=$(ls -l "$archivo" | awk '{print $1}')
echo -e "${negrita}Nombre:${nada} $nombre"
echo -e "${negrita}Tamaño:${nada} $tamano"
echo -e "${negrita}Dueño:${nada} $dueno"
echo -e "${negrita}Permisos:${nada} $permisos"
}

# Función para listar archivos en formato de tabla
listar_archivos() {
    directorio=$1
    cantidad=$2
    filtro=$3

    # Verifica si el argumento proporcionado es un archivo en lugar de un
    directorio
    if [ -f "$directorio" ]; then
        echo -e "${rojo}ERROR:${nada} Los parámetros -g, -p y -x solo aceptan
directorios, no archivos."
        mostrar_uso
    fi

    # Verifica si el directorio no existe
    if [ ! -d "$directorio" ]; then
```

```

    echo -e "${rojo}ERROR:${nada} La ruta: ${negrita}$directorio${nada} no
existe."

    echo -e Verifique la ${negrita}ruta${nada}.

    mostrar_uso
fi

# Si se solicitó mostrar solo archivos ejecutables, aplica un filtro
adicional en el comando find
if [ "$filtro" = "ejecutables" ]; then
    archivos=$(find "$directorio" -maxdepth 1 -type f -executable -printf
"%s %p\n")
else
    archivos=$(find "$directorio" -maxdepth 1 -type f -printf "%s %p\n")
fi

# Ordena los archivos por tamaño si se solicitan los más grandes o los
más pequeños
if [ -n "$archivos" ]; then
    if [ "$modo" = "grandes" ]; then
        archivos=$(echo "$archivos" | sort -nr | head -n "$cantidad")
    elif [ "$modo" = "pequenos" ]; then
        archivos=$(echo "$archivos" | sort -n | head -n "$cantidad")
    fi
fi

# Imprime los encabezados de la tabla con tamaño, nombre, dueño y
permisos
printf "%-10s %-40s %-10s %-10s\n" "Tamaño" "Nombre" "Dueño" "Permisos"
                                echo -e
${negrita}-----
-----${nada}

# Recorre los archivos obtenidos e imprime la información formateada
echo "$archivos" | while read -r tamano nombre; do
    if [ -z "$tamano" ] || [ -z "$nombre" ]; then

```

```

        continue
    fi

    dueno=$(ls -l "$nombre" | awk '{print $3}')
    permisos=$(ls -l "$nombre" | awk '{print $1}')
    nombre=$(basename "$nombre")
    tamaño=$(numfmt --to=iec "$tamaño")

    printf "%-10s  %-40s  %-10s  %-10s\n" "$tamaño" "$nombre" "$dueno"
"$permisos"
done
}

# Variables predeterminadas para la ruta y los modos de operación
ruta="."
modo=""
filtro=""

i=1
opciones_contadas=0

# Bucle para procesar los argumentos pasados al script
while [ "$i" -le "$#" ]; do
    arg=${!i}

    # Verifica el tipo de argumento que es y actúa en consecuencia
    case "$arg" in
        -s)

            # Verifica que la opción -s esté sola y no combinada con otras
opciones

            if [ "$i" -ne 1 ]; then

                echo -e "${rojo}ERROR:${nada} La opción -s no puede combinarse con
otras opciones."

                mostrar_uso
            fi

            i=$((i + 1))
            archivo=${!i}

```

```
# Verifica si no se proporcionó un archivo después de -s
if [ -z "$archivo" ]; then
    echo -e "${rojo}ERROR:${nada} Debe proporcionar un archivo después
de -s."

    mostrar_uso
fi
info_archivo "$archivo"
exit 0
;;
-g)
# Verifica que no se usen las opciones -g y -p juntas
if [ "$modo" = "pequenos" ]; then
    echo -e "${rojo}ERROR:${nada} No se pueden usar -g y -p al mismo
tiempo."

    mostrar_uso
fi
modo="grandes"
opciones_contadas=$((opciones_contadas + 1))
;;
-p)
# Verifica que no se usen las opciones -g y -p juntas
if [ "$modo" = "grandes" ]; then
    echo -e "${rojo}ERROR:${nada} No se pueden usar las opciones -g y
-p al mismo tiempo."

    mostrar_uso
fi
modo="pequenos"
opciones_contadas=$((opciones_contadas + 1))
;;
-x)
filtro="ejecutables"
opciones_contadas=$((opciones_contadas + 1))
;;
*)
```

```
# Asigna la ruta si el argumento no es una opción reconocida
if [[ "$arg" != -* ]]; then
    ruta="$arg"
else
    echo -e "${rojo}ERROR:${nada} El parámetro '${negrita}$arg${nada}'
no es válido."
    mostrar_uso
fi
;;
esac
i=$((i + 1))
done

# Verifica que no se hayan pasado más de dos opciones al mismo tiempo
if [ "$opciones_contadas" -gt 2 ]; then
    echo -e "${rojo}ERROR:${nada} Solo se permiten hasta 2 parámetros de
opción al mismo tiempo."
    mostrar_uso
fi

# Llama a la función de listar archivos, pasando la ruta y el modo si se
ha definido
if [ "$modo" = "grandes" ] || [ "$modo" = "pequenos" ]; then
    listar_archivos "$ruta" 3 "$filtro"
else
    listar_archivos "$ruta" 0 "$filtro"
fi
```